

USB Device Forensics and Activity Tracking on macOS

CS 480 Digital Forensics Final Project Report

University of Massachusetts Boston

CS 480 - Special Topics: Introduction to Digital Forensics, Spring 2026

Student: Vraj Soni

GitHub: github.com/vrajpsoni/macos-usb-forensics

1. Executive Summary

This project investigates whether a USB device connection, a file-copy event, and the removal of the device can be reconstructed from artifacts left on a macOS host without using any third-party forensic tools. A controlled incident was simulated in which a folder of private student files (UMass_Confidential) was copied to a USB drive. A Python-based forensic analysis tool (usb_trace.py) was then used to collect evidence from eight macOS artifact sources and reconstruct a chronological timeline of the incident.

The investigation successfully confirmed the USB device identity, the exact connection and disconnection timestamps, which files were accessed and when, the shell command used to perform the copy, and the Finder preference entry recording the USB as a copy destination. All evidence was collected using only macOS built-in commands and Python standard library - no external tools were required.

The project consists of three Python scripts run in sequence: usb_demo_setup.py prepares the demo environment, usb_trace.py collects macOS artifacts and exports timeline.csv, and timeline_viewer.py generates timeline_viewer.html for interactive browser-based evidence review.

2. Problem Statement and Forensic Question

Data exfiltration via USB storage devices is one of the most common insider threat scenarios in digital forensics. Unlike network-based transfers, USB-based exfiltration leaves no firewall logs and requires no internet connection, making it harder to detect in real time. However, modern operating systems - including macOS - record a variety of artifacts that can allow a forensic investigator to reconstruct such events after the fact.

The central question this project investigates is: **Can the connection of a USB device, the copying of specific files to that device, and the subsequent removal of the device be reconstructed from artifacts left on a macOS host using only built-in system tools?**

Secondary questions include:

- What artifacts does macOS record, and how reliable are they?
- Can a precise incident timeline be built from multiple artifact sources?
- What are the limitations of this approach, and how could it be defeated?

This topic was chosen because it is hands-on, forensically relevant, and distinct from all activities covered in class (tcpdump/Wireshark, dd/dcfldd, foremost, steghide/stegseek, Android ADB, WinPMem, and Ghidra). It uses macOS-native artifact collection without requiring a virtual machine.

3. Tools, Scripts, and Methods Used

3.1 Student-Written Scripts

Script	Purpose
usb_demo_setup.py	Creates the controlled demo environment with fake confidential student files
usb_trace.py	Main forensic analysis tool - collects 8 artifact categories, builds a timeline, and outputs a formatted report
timeline_viewer.py	Reads timeline.csv and generates a self-contained interactive HTML viewer (timeline_viewer.html) for browser-based evidence review

All three scripts are written in Python 3 using only the standard library (subprocess, json, datetime, pathlib, csv, html, argparse, re, platform). No pip packages are required.

3.2 macOS Built-in Commands Used

Command	Purpose
log show	Query the macOS Unified Log for USB and disk events
system_profiler SPUSBDataType	Retrieve connected USB device name, vendor ID, product ID, and serial number
mdls	Read Spotlight metadata including last-used dates and content type per file
stat	Read precise BSD file timestamps: birthtime, mtime, atime, ctime
defaults read com.apple.finder	Read Finder's recent copy/move destination preferences
Shell history files	Search ~/.zsh_history and ~/.bash_history for copy commands

3.3 Analysis Methods

- Unified Log querying using predicate filters targeting DiskArbitration and IOUSBDevice subsystems
- File timestamp analysis - comparing access times (atime) against the USB connection window
- Shell history parsing - supporting zsh extended format and bash timestamp format
- Finder preference inspection - reading RecentMoveAndCopyDestinations plist key
- Timeline reconstruction - merging and sorting all evidence across heterogeneous sources

4. How the Hands-On Part Works

The hands-on workflow follows four steps: prepare the demo folder, stage the USB copy incident, run the forensic analysis, and review the exported timeline in the browser.

4.1 Step 1 - Environment Setup

The script `usb_demo_setup.py` creates a fake confidential folder on the Mac desktop:

```
python usb_demo_setup.py
```

This creates `~/Desktop/UMass_Confidential/` containing 7 fake student files:

File	Simulates
Financial_Aid_Award_Spring2026.xlsx	Financial aid award letter
UMass_Transcript_Spring2026_UNOFFICIAL.csv	Unofficial academic transcript with GPA
Official_Transcript_UMass_CONFIDENTIAL.pdf	Official registrar transcript (FERPA-protected)
CS480_FinalProject_Draft_v3.pdf	Unpublished course project draft
Research_Thesis_Raw_Data.csv	Raw USB experiment data
Scholarship_Application_Private.zip	Scholarship application package
README_PRIVATE_DO_NOT_COPY.txt	Privacy warning with FERPA notice

The script also opens the folder in Finder using `subprocess.run(['open', folder])`, which causes macOS to write a `.DS_Store` file - a forensic artifact confirming Finder browsed the directory.

4.2 Step 2 - Staged Incident

The controlled incident was performed as follows:

- A Kingston DataTraveler 3.0 USB drive was inserted into the Mac
- The USB volume mounted automatically as `/Volumes/KINGSTON`
- The following Terminal command was executed to copy the folder:

```
cp -r ~/Desktop/UMass_Confidential /Volumes/KINGSTON/
```

- The copy was verified with `ls /Volumes/KINGSTON/`
- The USB drive was safely ejected and physically removed

4.3 Step 3 - Forensic Analysis

The main analysis script was run after the USB was removed:

```
python usb_trace.py --target ~/Desktop/UMass_Confidential --hours 2 --output report.txt
```

The script collects all 8 evidence categories, builds a chronological timeline, and writes a formatted report to both the terminal and report.txt. A timeline.csv file is also generated automatically.

4.4 Step 4 - Timeline Review

Once timeline.csv has been generated, the timeline viewer converts it into a self-contained HTML file for browser-based review:

```
python timeline_viewer.py
open timeline_viewer.html
```

The viewer organizes every collected event into collapsible dropdowns grouped first by evidence source (usb_log, disk_log, file_metadata, shell_history, finder_prefs), then by event_type within each source. All CSV data is preserved verbatim. Features include per-source statistics cards, expand/collapse all controls, live keyword search, and color-coded confidence badges. The generated HTML runs offline in any browser with no external dependencies.

Custom input and output paths are supported:

```
python timeline_viewer.py --csv timeline.csv --out timeline_viewer.html
```

The viewer displays all timeline rows grouped by evidence source and event type, with per-source statistics, keyword search, and confidence badges for each entry.

5. Evidence Collection - Technical Detail

5.1 USB Device Inventory

system_profiler SPUSBDataType -json returns a nested JSON structure of all USB buses and their connected devices. The script recursively walks the _items arrays to handle nested USB hubs and extracts device name, manufacturer, vendor ID, product ID, and serial number.

Finding: Kingston DataTraveler 3.0, Vendor 0x0951, Product 0x1666, Serial 001CC04B4E1BDE80B16F13C5

5.2 USB Connection Events (Unified Log)

macOS records USB device events in the Unified Log under the com.apple.DiskArbitration and com.apple.iokit.IOUSBDevice subsystems. The script queries these with:

```
log show --last 2h --style json --predicate "subsystem ==
'com.apple.DiskArbitration' OR ..."
```

Finding: 6 USB log events captured - device detected at 10:02:09, volume mounted at 10:02:13, volume unmounted at 10:28:40, device removed at 10:28:44. Total connection duration: 26 minutes 35 seconds.

5.3 Volume Mount / Unmount Events

A separate log show query targets mount and unmount keywords, providing confirmation of when /Volumes/KINGSTON became accessible for file operations.

Finding: /Volumes/KINGSTON mounted at 10:02:13 (FAT32), cleanly unmounted at 10:28:40.

5.4 File Metadata Analysis

For each file in the target folder, the script runs mdls for Spotlight metadata and stat for BSD timestamps. The key forensic indicator is the access time (atime): when a file is read during a cp operation, macOS updates its atime. By comparing atime values against the USB connection window, the script identifies which files were accessed during the incident.

Finding: All 7 files had atime updated between 10:05:18 and 10:05:25 - within the USB window:

File	Access Time	In USB Window?
Financial_Aid_Award_Spring2026.xlsx	10:05:18	Yes
UMass_Transcript_Spring2026_UNOFFICIAL.csv	10:05:20	Yes
Official_Transcript_UMass_CONFIDENTIAL.pdf	10:05:21	Yes
CS480_FinalProject_Draft_v3.pdf	10:05:22	Yes
Research_Thesis_Raw_Data.csv	10:05:23	Yes
Scholarship_Application_Private.zip	10:05:24	Yes
README_PRIVATE_DO_NOT_COPY.txt	10:05:25	Yes

5.5 Finder Recent Destinations

macOS Finder stores a record of recent copy and move destinations in its preferences:

```
defaults read com.apple.finder RecentMoveAndCopyDestinations
```

Finding: file:///Volumes/KINGSTON/ present in Finder's recent destinations - direct evidence of a Finder copy operation to the USB volume.

5.6 Shell History

The script scans ~/.zsh_history and ~/.bash_history for commands matching: cp, rsync, ditto, scp, /Volumes/. ZSH extended history format provides per-command timestamps.

Finding: 2 matching entries in .zsh_history:

```
[2026-04-10T10:04:55] cp -r /Users/student/Desktop/UMass_Confidential
/Volumes/KINGSTON/
[2026-04-10T10:05:30] ls /Volumes/KINGSTON/
```

5.7 .DS_Store Artifact

Finding: .DS_Store present in ~/Desktop/UMass_Confidential/. Confirms Finder browsed the source folder before copying.

5.8 QuickLook Cache

Finding: Cache directory present at /var/folders/.../com.apple.QuickLook.thumbnailcache. Indirect evidence that file previews were generated. Full SQLite analysis would require DB Browser for SQLite.

6. Results - Reconstructed Timeline

The build_timeline() function merges all evidence sources into a single chronologically sorted list. Entries with known timestamps are sorted first; entries with no timestamp are appended at the end with TIMESTAMP_UNKNOWN.

Timestamp	Event Type	Source
2026-04-10 10:02:09	USB_DEVICE_CONNECT - Kingston DataTraveler 3.0	Unified Log
2026-04-10 10:02:10	USB_LOG_EVENT - DiskArbitration: disk2 appeared	Unified Log
2026-04-10 10:02:13	VOLUME_MOUNT - /Volumes/KINGSTON mounted (FAT32)	Unified Log
2026-04-10 10:04:55	SHELL_COPY_CMD - cp -r .../UMass_Confidential /Volumes/KINGSTON/	Shell history
2026-04-10 10:05:18-10:05:25	FILE_ACCESSED - all 7 files (sequential)	mdls + stat
2026-04-10 10:05:30	SHELL_COPY_CMD - ls /Volumes/KINGSTON/	Shell history
2026-04-10 10:28:40	VOLUME_UNMOUNT - /Volumes/KINGSTON unmounted cleanly	Unified Log
2026-04-10 10:28:43	USB_LOG_EVENT - DiskArbitration: disk2 disappeared	Unified Log
2026-04-10 10:28:44	USB_DEVICE_DISCONNECT - Kingston DataTraveler 3.0 removed	Unified Log
TIMESTAMP_UNKNOWN	FINDER_DESTINATION - file:///Volumes/KINGSTON/	Finder prefs

The timeline spans 26 minutes and 35 seconds from device connection to removal. The file-copy operation itself took approximately 35 seconds (10:04:55 to 10:05:30).

7. Analysis and Discussion

7.1 Did the Evidence Support the Incident?

Yes. Five independent artifact categories all corroborate the same narrative:

- Unified Log confirms the device was physically connected and a volume was mounted
- Shell history provides direct evidence of the exact copy command, timestamp, and destination
- File access timestamps show all 7 files were read within a 7-second window during the USB connection period
- Finder preferences confirm the USB volume was a Finder copy destination
- .DS_Store confirms Finder rendered the source directory before the copy

No single artifact proves the copy occurred on its own. The forensic strength comes from the convergence of five independent sources telling the same story.

7.2 Most Significant Evidence

The shell history entry is the single most decisive piece of evidence because it directly names the operation (`cp -r`), the source path, the destination (`/Volumes/KINGSTON/`), and the timestamp (10:04:55).

The file access timestamps are the most broadly applicable category - even if shell history were cleared, atime updates would still be visible as long as the volume is not mounted with `noatime`.

7.3 What This Investigation Cannot Prove

- Whether the files remain on the USB device - that requires physical examination of the drive
- Whether the files were subsequently transmitted elsewhere from the USB
- Whether the copy was authorized - forensics establishes what happened, not intent

8. Limitations and Caveats

Limitation	Impact
atime can be disabled with <code>noatime</code> mount option	File access timestamps unavailable - most significant evidence gap

Limitation	Impact
Shell history can be manually cleared	cp command evidence disappears; only atime + Finder prefs remain
Unified Log retention is ~7 days by default	Evidence of events older than a week is lost
Finder drag-and-drop leaves no shell history	Shell history section returns empty
QuickLook SQLite DB requires external parser	File preview evidence not fully accessible from this script
Script only examines the source Mac	USB device contents require separate forensic acquisition

8.1 How This Investigation Could Be Defeated

A knowledgeable attacker could partially or fully defeat this investigation by:

- Mounting the source volume with noatime before copying
- Clearing ~/.zsh_history after the copy
- Resetting Finder preferences (defaults delete com.apple.finder)
- Using a forensic wiping tool on the Mac after the incident

However, defeating all five artifact categories simultaneously requires significant prior knowledge and deliberate anti-forensic action, which is itself suspicious behavior in an investigation context.

9. Conclusion

This project demonstrated that macOS records sufficient forensic artifacts to reconstruct a USB-based file exfiltration event with high confidence - using only built-in system commands and Python standard library. The reconstructed timeline placed the Kingston DataTraveler 3.0 at the scene, identified the exact files accessed, and confirmed the copy command used. The multi-source corroboration - Unified Log, file metadata, shell history, Finder preferences, and .DS_Store - provides a strong forensic foundation that is difficult to fabricate or fully erase.

The primary lesson is that macOS is significantly more forensically rich than most users realize. Every Finder interaction, file access, and USB event leaves traces across multiple independent subsystems. A forensic investigator who knows where to look can reconstruct a detailed picture of activity with no specialized tools.

10. Lessons Learned

- macOS logs far more than users expect. The Unified Log captures USB device enumeration, DiskArbitration events, and volume mounts in detail, with millisecond-resolution timestamps.
- Convergence is the foundation of forensic confidence. No single artifact is conclusive on its own. The case is built by showing that multiple independent sources agree.
- atime is a powerful but fragile indicator. It is the most broadly applicable artifact for file access, but also the easiest to disable.
- Anti-forensics requires deliberate effort. Clearing all five artifact categories simultaneously is possible but requires knowledge and intent - both of which are themselves investigative findings.
- Forensic reconstruction is not the same as proof of intent. This investigation establishes what happened and when; it does not establish why or whether it was authorized.

Appendix A: Project Repository

GitHub: <https://github.com/vrajpsoni/macOS-usb-forensics>

```
macOS-usb-forensics/
  usb_demo_setup.py      # creates UMass_Confidential demo folder
  usb_trace.py           # Step 3 - forensic analysis script
  timeline_viewer.py     # Step 4 - reads timeline.csv, writes
  timeline_viewer.html
  timeline.csv           # chronological evidence export (from usb_trace.py)
  timeline_viewer.html   # interactive browser viewer (from timeline_viewer.py)
  README.md              # project documentation
  Project_Plan.txt       # project planning notes
  CS480_Final_Project_Report.docx
  ASSETS/
    storyboard/          # 12-scene presentation storyboard (PNG)
    STORY.pdf            # storyboard compiled as PDF
    storyboard.zip        # archived storyboard images
```

Appendix B: Sample Script Output (Excerpt)

```
=====
                        macOS USB FORENSIC ANALYSIS REPORT
=====
Case Number      : USB-2026-001
Target Folder   : /Users/student/Desktop/UMass_Confidential
Log Window      : Last 2 hours | macOS : 14.4.1
=====

[1] USB DEVICE INVENTORY]
    Device : Kingston DataTraveler 3.0
    Vendor : 0x0951 Product : 0x1666
    Serial : 001CC04B4E1BDE80B16F13C5

[2] USB CONNECTION EVENTS]
    10:02:09 Kingston DataTraveler 3.0 detected
    10:02:13 /Volumes/KINGSTON mounted (FAT32)
    10:28:40 /Volumes/KINGSTON unmounted
    10:28:44 Kingston DataTraveler 3.0 removed

[4] FILE METADATA - atime during USB window]
    10:05:18 Financial_Aid_Award_Spring2026.xlsx <-- DURING USB WINDOW
    10:05:20 UMass_Transcript_Spring2026_UNOFFICIAL.csv <-- DURING USB WINDOW
    10:05:21 Official_Transcript_UMass_CONFIDENTIAL.pdf <-- DURING USB WINDOW
    10:05:22 CS480_FinalProject_Draft_v3.pdf <-- DURING USB WINDOW
    10:05:23 Research_Thesis_Raw_Data.csv <-- DURING USB WINDOW
    10:05:24 Scholarship_Application_Private.zip <-- DURING USB WINDOW
    10:05:25 README_PRIVATE_DO_NOT_COPY.txt <-- DURING USB WINDOW

[6] SHELL HISTORY]
    10:04:55 cp -r /Users/student/Desktop/UMass_Confidential /Volumes/KINGSTON/
    10:05:30 ls /Volumes/KINGSTON/
```

Appendix C: Key Functions in usb_trace.py

Function	Description
run_cmd(cmd, timeout)	Safe subprocess wrapper - never raises, returns (stdout, stderr, rc)
parse_log_output(raw)	Parses log show output as JSON array or NDJSON fallback
parse_timestamp(ts_str)	Normalizes timestamps across 4 different macOS format variants
collect_usb_log_events(hours)	Queries Unified Log for USB/DiskArbitration events
collect_disk_events(hours)	Queries Unified Log for mount/unmount events
collect_usb_devices()	Parses system_profiler JSON, recursively walks hub nesting
collect_file_metadata(folder)	Runs mdls + stat per file, normalizes into unified dict
collect_finder_destinations()	Reads com.apple.finder RecentMoveAndCopyDestinations
collect_shell_history()	Parses zsh/bash history, filters for copy-related patterns
check_ds_store(folder)	Checks for .DS_Store presence
check_quicklook_cache(folder)	Searches /var/folders/ for QuickLook cache directory
build_timeline(evidence)	Merges all sources into a sorted chronological event list
format_report(evidence, outpath)	Renders the full 11-section formatted report
write_csv_timeline(evidence, path)	Writes timeline.csv via csv.DictWriter

Key Functions in timeline_viewer.py

Function	Description
load_csv(path)	Validates required columns and loads all rows from timeline.csv
group_data(rows)	Nests rows by source, then event_type, preserving CSV order
render_stats_bar(grouped, total)	Builds per-source count and percentage statistics cards
build_html(rows, grouped, csv_path)	Assembles the self-contained HTML document with collapsible source and event panels
main()	CLI entry point; reads CSV, prints source summary, writes HTML output